

Meeting Scribe — Product Requirements Document

A local-first, bot-free meeting transcription & document generation app for Windows

Document	Product Requirements Document (PRD) v1.0
Product	Meeting Scribe — Local Meeting Assistant
Author	Areloa
Date	May 2026
Status	Draft — for review
Target OS	Microsoft Windows 10/11 (x64)

1. Executive Summary

Meeting Scribe is a standalone desktop application for Windows that silently listens to any meeting taking place on the user's computer, transcribes it in real time, and generates polished output documents (Minutes of Meeting, action item lists, timelines, custom-template reports) — all without a bot ever joining the call.

The product is engineered around three uncompromising principles: it must run fully on-device for privacy, it must be free to operate using local open-source models, and it must travel with the user — project files are stored as portable bundles that can be opened from any device the user logs into.

Problem

Existing meeting assistants either (a) require a bot to join the call, which is awkward, frequently blocked by IT, and visible to other attendees, or (b) ship audio to a vendor's cloud, raising privacy concerns and forcing the user onto a paid subscription once usage grows beyond a generous-looking free tier.

Solution

A single Windows installer that captures both the user's microphone and the system's loopback audio (WASAPI), transcribes locally with Whisper, and renders documents from user-defined templates. The app is free, offline-capable, and never exposes meeting content to an external service unless the user explicitly opts in.

Why this is different

- No bot. No meeting invite. Other participants never see the assistant.
- Fully local by default — Whisper for transcription, Ollama / Llama for document drafting.
- Free forever for personal use — no per-minute caps, no subscription gating core features.
- User-defined templates: any .docx with placeholders becomes a reusable output format.
- Portable project bundles — drop them in OneDrive / Dropbox / a USB key and resume on another machine.

2. Goals & Non-Goals

Goals (in scope)	Non-goals (out of scope for v1)
• Capture mic + system audio on Windows. • Transcribe locally with speaker diarization. • Allow user to define output templates. • Generate MoM, action items, timeline, custom docs. • Persist audio + transcript + docs in a portable project bundle. • Cross-device continuity via user-chosen sync folder.	• Bots that join Zoom / Teams / Meet as a participant. • Real-time coaching during the meeting. • Mobile (Android / iOS) — desktop only in v1. • Multi-tenant SaaS or shared workspaces. • CRM integrations (Salesforce, HubSpot). • Live translation between languages in v1.

3. Target Users & Personas

Persona	Primary need	How Meeting Scribe helps
Product / Project Manager	Reliable minutes of meeting with action items and owners.	Custom MoM template auto-filled after each call; tasks extracted with owners.
Engineer / Tech Lead	Timeline of technical decisions made in design reviews.	Timeline view + searchable transcript stored locally with the codebase.
Consultant / Solo founder	Client-deliverable documentation without cloud exposure.	Branded .docx templates rendered locally; NDAs respected — audio never leaves device.
Student / Researcher	Free, unlimited transcription of lectures and interviews.	Unlimited local Whisper transcription with zero subscription cost.

4. Competitive Landscape

The meeting-assistant market is crowded but converges on a small set of architectural choices: send audio to a vendor's cloud, transcribe there, and bill per minute. Meeting Scribe deliberately makes the opposite choice on every axis where privacy, cost or portability is in tension with vendor convenience.

Capability	Meeting Scribe (this app)	Otter.ai	Fireflies.ai	MS Teams Live	Granola
Bot joins meeting	No	Yes	Yes	N/A	No
Works offline	Yes	No	No	Limited	No
Audio stays on device	Yes	No (cloud)	No (cloud)	No (cloud)	Partial
Free-tier minutes/month	Unlimited	300 min	Limited	Requires E5	~25 meetings
Custom .docx templates	Yes	Limited	Limited	No	Limited
Speaker diarization	Yes (local)	Yes	Yes	Yes	Yes
Cross-device continuity	Yes (own folder)	Yes (vendor cloud)	Yes (vendor cloud)	Yes	Yes
Cost per meeting at scale	\$0	Paid above 300m	Paid	M365 license	Paid
Vendor lock-in	None — open files	Yes	Yes	Yes	Yes

Capability	Meeting Scribe (this app)	Otter.ai	Fireflies.ai	MS Teams Live	Granola
Source-controllable	Yes (.docx + .json)	No	No	No	No

Figures for competitors are approximate and based on publicly listed plans as of May 2026; users should verify before contractual commitments.

Why this matters

- Privacy: meetings often include unreleased product details, salaries, customer PII. Cloud transcription puts them in someone else's database.
- Cost predictability: bot-based tools price per minute or per seat, which scales linearly with usage. A local app has zero marginal cost.
- Behavior: bots change the meeting. People speak differently when an unfamiliar attendee labelled 'Otter' is in the call.
- Portability: vendor clouds make export friction-y. Local files are yours to move, archive, or version-control.

5. Functional Requirements

Requirements are tagged Must / Should / Nice using MoSCoW prioritisation.

ID	Requirement	Priority
FR-01	Capture microphone and system loopback audio simultaneously on Windows (WASAPI).	Must have
FR-02	Run end-to-end fully offline; no internet connection required for core features.	Must have
FR-03	Transcribe captured audio locally using Whisper (faster-whisper) with selectable model size.	Must have
FR-04	Persist audio file, raw transcript, and metadata as a single portable project bundle (.mscribe).	Must have
FR-05	Allow the user to define document templates (.docx) with named placeholders that the app fills in.	Must have
FR-06	Generate Minutes of Meeting, action items, and a meeting timeline from any session.	Must have
FR-07	Speaker diarization — distinguish between speakers in the transcript.	Must have
FR-08	Pause and resume recording at any time during a session.	Must have
FR-09	Project bundles can be saved to any folder the user designates, including OneDrive / Dropbox, so the user can resume on another machine.	Must have
FR-10	Provide a local LLM option (Ollama) for document drafting, with an opt-in BYO-	Should have

ID	Requirement	Priority
	API-key path for Claude / OpenAI.	
FR-11	Detect topic shifts and tag transcript sections with a phase label (intro, discussion, decision, action).	Should have
FR-12	Export documents to .docx and .pdf.	Should have
FR-13	Full-text search across past meetings stored locally.	Should have
FR-14	Template marketplace / starter pack: ship with MoM, 1:1 recap, interview notes, standup, and timeline templates.	Should have
FR-15	Redaction of sensitive terms (regex + named-entity) before document generation.	Nice to have
FR-16	Audio playback synced to transcript (click a line, play that moment).	Nice to have
FR-17	Optional end-to-end encryption of project bundles at rest.	Nice to have
FR-18	Multi-language transcription (Whisper supports ~100 languages out of the box).	Nice to have

6. Non-Functional Requirements

Category	Target
Performance	Whisper-small transcription must keep pace with real-time on a 4-core x64 CPU. End-to-end document generation under 60s for a 30-minute meeting.
Privacy	Zero outbound network traffic by default. Any opt-in cloud usage must be behind an explicit toggle with a confirmation dialog.
Portability	Project bundles are self-contained ZIP files using open formats — audio (.wav/.opus), transcript (.json), generated docs (.docx/.pdf), metadata (.json).
Installer footprint	Under 500 MB installed, including the small Whisper model. Larger models downloaded on demand.
Reliability	Crash during recording must not lose audio captured so far — write to disk in rolling chunks.
Accessibility	Keyboard navigation for all core actions; high-contrast theme; screen-reader-friendly transcript view.

7. System Architecture

Audio flows through seven deterministic stages from microphone to finished document. Each stage is independently testable and replaceable.

1	CAPTURE	Microphone via sounddevice; system audio via WASAPI loopback. Streams are mixed into a single 16 kHz mono buffer and written to disk as rolling chunks so nothing is lost on crash.
2	SEGMENT	Voice activity detection (silero-vad) splits the buffer into utterances. Empty / silent regions are skipped to save compute.
3	TRANSCRIBE	faster-whisper runs locally (small / medium / large depending on user's hardware). Output is timestamped tokens.
4	DIARIZE	pyannote.audio assigns a speaker label to each utterance. Labels can later be renamed by the user (e.g. Speaker 1 → 'Sarah').
5	STRUCTURE	A local LLM (Ollama / Llama 3.1 8B) classifies utterances into phases (intro, discussion, decision, action) and extracts action items with owners.
6	RENDER	User-selected .docx template is filled in with structured data via docxtpl. Documents can be re-rendered without re-running transcription.
7	PERSIST	Everything (audio, transcript, structured data, generated docs) is packaged into a .mscribe bundle and saved to the user's chosen folder.

Local-first data flow

Stages 1–3 must run for a meeting to be useful. Stages 4–6 are deferred work that happens after the meeting ends, so they don't compete with transcription for CPU. Stage 7 is continuous — the project bundle is updated on every meaningful change.

8. Tech Stack

Every component listed has a free tier sufficient for unlimited personal use. There is no paid dependency on the critical path.

Component	Purpose	Tool	Cost
Audio capture	Mic + system loopback on Windows	sounddevice + WASAPI	Free
VAD	Skip silence	silero-vad	Free
Transcription	Audio → text, runs locally	faster-whisper	Free
Diarization	Speaker labels	pyannote.audio	Free
Local LLM	Document drafting offline	Ollama + Llama 3.1 8B	Free
Cloud LLM (opt-in)	Higher-quality drafting	Claude API (BYO key)	BYO
Template engine	Fill user .docx templates	docxtpl + python-docx	Free
PDF export	Render docs as PDF	WeasyPrint / docx2pdf	Free

Component	Purpose	Tool	Cost
Local store	Index meetings for search	SQLite + FTS5	Free
UI	Desktop app shell	PyQt6 (Qt for Python)	Free
Packaging	Windows installer	PyInstaller + Inno Setup	Free
Sync (user-owned)	Bundles saved to OneDrive / Dropbox	Native folder sync	Free

Estimated cost to operate the app indefinitely on a single device: \$0.

9. UI / UX Specification

The app uses a Files-style master/detail layout. The recording bar is always-on-top and intentionally minimal so the user can keep it visible during the meeting without distraction.

Screen / Panel	Content	Behavior
Home	List of past meetings (title, date, duration, primary speaker). Big 'Start new meeting' button.	Click any row to open the meeting workspace.
Recording bar	Compact floating window: REC indicator, waveform, elapsed time, pause / stop, audio source selector.	Always-on-top. Can be moved to a second monitor. Hidden from screen-shares optionally.
Meeting workspace	Three panes: transcript (left), structured data view — action items, decisions, timeline (middle), output documents (right).	Editable transcript with speaker re-labelling. Re-generate docs without re-transcribing.
Templates manager	Browse, import, duplicate, edit .docx templates and their placeholder schema.	'Test render' button shows what the template would produce with sample data.
Settings	Default project folder, Whisper model size, LLM backend (local / Ollama / Claude), redaction rules.	Changing the project folder migrates the index but never moves user-owned bundles automatically.

10. User-Defined Template System

Templates are the keystone feature that turns Meeting Scribe from a transcription tool into a document factory. Users define what they want out of each meeting; the app fills the blanks.

How a template is defined

1. User creates a normal Word document with their desired layout and branding.
2. Wherever they want generated content, they insert a Jinja placeholder, e.g. `{{ meeting.title }}`, `{{ action_items }}`, `{{ timeline }}`.
3. They drop the .docx into the Templates folder (or import via the UI).

4. The app parses placeholders, validates against the schema, and shows a 'Test render' preview.

Built-in placeholder schema

- **meeting.title** — string, the user-set or auto-detected title.
- **meeting.date** — ISO 8601 date of the session.
- **meeting.duration** — human-readable duration.
- **attendees** — list of speaker labels (renamed by the user).
- **summary** — short executive summary written by the local LLM.
- **decisions** — list of decisions and the speaker who made them.
- **action_items** — list of {owner, description, due_date}.
- **timeline** — chronological list of {timestamp, phase, topic}.
- **transcript** — full transcript array of {speaker, start, text}.

Shipped starter templates

- Minutes of Meeting (formal)
- 1:1 recap
- Standup digest
- Interview notes
- Decision log
- Timeline / event recap

11. Data Model & Cross-Device Continuity

A meeting is materialised as a .mscribe bundle — a ZIP archive whose internal layout is fully documented and stable across versions, so users (and other tools) can rely on its contents.

Bundle layout

- **audio.opus** — compressed source audio.
- **transcript.json** — speaker-labelled timestamped utterances.
- **structured.json** — decisions, action items, timeline, summary.
- **documents/** — generated .docx and .pdf files.
- **meta.json** — title, date, duration, app version, template used.

Cross-device continuity

The app does not run its own cloud. Instead, the user picks a default project folder that already syncs (OneDrive, Google Drive, Dropbox, or a network share). On a second machine, the user installs the app, points it at the same folder, and the local SQLite index rebuilds from the bundles found there. There is no account, no login, and no proprietary sync server — but the user can pick up exactly where they left off.

If the user starts a recording on machine A and then opens machine B, an active session lock inside the bundle prevents concurrent edits. Documents can be regenerated independently on either machine because raw audio and transcript travel with the bundle.

12. Risks & Mitigations

Severity	Risk & mitigation
HARD	Whisper performance on low-end CPUs On a 2-core machine, large Whisper models cannot keep up with real time. Mitigation: ship with whisper-small as default; only enable larger models after a hardware probe at first launch.
HARD	System-audio capture permissions On Windows, WASAPI loopback works without a driver, but some pro audio interfaces (ASIO) bypass it. Mitigation: document supported interfaces; provide a fallback to capture from a virtual cable (VB-CABLE) for edge cases.
MEDIUM	Local LLM quality vs. cloud LLM Llama 3.1 8B produces good but inferior summaries compared to Claude. Mitigation: BYO-key path for users who want quality, with explicit consent dialog on first use.
MEDIUM	Bundle size on sync drives 30-minute meetings produce ~25 MB bundles (mostly audio). 100 meetings = 2.5 GB. Mitigation: setting to store audio outside the bundle, or auto-prune audio after N days.
MEDIUM	Concurrent edits across devices User opens the same bundle on two machines. Mitigation: file-lock metadata inside the bundle; warn on open if another instance has it locked.
LOW	PyInstaller bundle size PyQt6 + faster-whisper + pyannote can hit ~400 MB. Mitigation: --onedir mode, ship small models by default, lazy-download larger ones.
LOW	User installs without admin rights WASAPI loopback works without admin; Inno Setup supports per-user install. Mitigation: prefer per-user install by default.

13. Build Plan

Seven modules, each independently testable. Don't start the next module until the current one demonstrably works.

1	Audio Capture ~2 days Capture mic + system audio simultaneously on Windows using sounddevice and WASAPI loopback. Mix into a single mono stream, write to disk in rolling chunks. Goal: 10-minute test recording with no dropouts. <i>Tools: sounddevice, pyaudiowpatch, numpy, soundfile</i>
2	Local Transcription ~3 days

	Wire faster-whisper to the captured audio. Add silero-vad for utterance segmentation. Test small / medium / large models for accuracy vs latency. Goal: real-time transcript dumped to terminal. <i>Tools: faster-whisper, silero-vad, ctranslate2</i>
3	Diarization & Structure ~3 days Add pyannote.audio for speaker labels. Wire Ollama with Llama 3.1 8B to extract action items, decisions, timeline from the transcript. Goal: a structured.json that captures who said what and what to do next. <i>Tools: pyannote.audio, ollama, langchain</i>
4	Template System ~2 days Implement docxtpl-based template rendering. Define the placeholder schema. Build a Templates folder watcher. Ship 5 starter templates. Goal: select a template, click render, get a finished .docx. <i>Tools: docxtpl, python-docx, jinja2</i>
5	Project Bundles & Storage ~2 days Define the .mscribe bundle format. Build the SQLite index for search. Implement open / save / autosave. Goal: closing the app and reopening reloads the same workspace state. <i>Tools: zipfile, sqlite3 (FTS5)</i>
6	PyQt6 UI ~4 days Build Home, Recording bar, Meeting workspace, Templates manager, Settings screens. Wire all backend services. Goal: end-to-end UX from 'New meeting' to 'Open finished MoM in Word'. <i>Tools: PyQt6, qt-material</i>
7	Packaging & Polish ~2 days PyInstaller --onedir build. Inno Setup installer with per-user install option. First-run hardware probe to pick Whisper model. Goal: a working .exe installer < 500 MB. <i>Tools: PyInstaller, Inno Setup</i>

Suggested timeline

Approximately 4–5 weekends if working solo evenings and weekends. Modules 1–3 are the technically risky ones; UI work is more predictable.

14. Success Metrics

Metric	Target
Transcription accuracy	Word Error Rate (WER) ≤ 10% on clear English audio with whisper-small.
End-to-end latency	From 'Stop recording' to 'MoM document ready' in under 60 s for a 30-minute meeting.
Privacy guarantee	Zero outbound network packets during a recorded session (verifiable via

Metric	Target
	Wireshark).
Cost per meeting	\$0 with default local backend. Optional cloud backend itemised per session.
User satisfaction	≥ 80% of generated MoMs require no manual edits before sharing.

15. Open Questions & Future Work

- Should v2 support macOS? (Requires BlackHole virtual driver — adds a setup step.)
- Mobile companion app for browsing past meetings on the go — out of scope for v1, but the bundle format is mobile-ready.
- Marketplace for community-contributed templates (curated, signed).
- Plugin system to let other apps consume the structured.json (e.g. push action items into Linear / Jira / Notion).
- Live translation for international meetings — Whisper supports the underlying capability already.

Appendix — Why this PRD improves on the original draft

Compared to the Sales Copilot PRD provided as input, this document strengthens the proposal in the following ways:

- Reframes the product around the user's stated needs (transcription + document generation) rather than real-time coaching, which was the original draft's focus.
- Replaces the cloud transcription dependency (Deepgram) with a local Whisper pipeline so the app is genuinely standalone and free.
- Adds a first-class user-defined template system with a documented placeholder schema.
- Defines a portable .mscribe bundle format so the user can move between devices without a proprietary cloud.
- Adds a full competitive landscape table benchmarking against Otter, Fireflies, Teams Live, and Granola.
- Expands risks beyond packaging to cover the harder issues: local model performance, audio capture edge cases, and concurrent device usage.
- Explicit personas, goals/non-goals, and success metrics — making scope easier to defend during build.